

Experiment 9 – DQN

NAME:N.AKSHAYA

REG.NO:192425129

COURSE CODE:MLA0305

COURSE NAME:REINFORCEMENT LEARNING

SOLT:B

9.Deep Q-Network (DQN)Implementation using TensorFlow and Keras.

Aim:

To implement a **Deep Q-Network (DQN)** using **TensorFlow and Keras** and analyze the agent's learning performance through reward and loss visualizations.

Procedure:

1. Import NumPy, TensorFlow, Keras, and Matplotlib libraries.
2. Define the state size and action size.
3. Build a DQN using Keras Dense layers.
4. Compile the network using the Adam optimizer and MSE loss.
5. Generate states and select actions using the DQN model.
6. Calculate rewards and update the Q-values.
7. Train the model for multiple episodes using `train_on_batch()`.
8. Store the reward and loss values for each episode.
9. Display the **DQN summary output**.
10. Plot reward and loss graphs to visualize the learning performance.

Python code:

```
import numpy as np, tensorflow as tf, matplotlib.pyplot as plt
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
model=Sequential([Dense(16,activation='relu',input_shape=(4,)),Dense(16,activation='r
elu'),Dense(2)])
```

```
model.compile(optimizer='adam',loss='mse')
```

```
rewards=[]; losses=[]
```

```
for ep in range(50):
```

```
    s=np.random.rand(4); total=0
```

```
    for _ in range(10):
```

```
        a=np.argmax(model.predict(s.reshape(1,-1),verbose=0)[0])
```

```
        ns=np.random.rand(4); r=1 if a==1 else -1
```

```
        q=model.predict(s.reshape(1,-1),verbose=0)
```

```
        q[0,a]=r+0.9*np.max(model.predict(ns.reshape(1,-1),verbose=0)[0])
```

```
        loss=model.train_on_batch(s.reshape(1,-1),q)
```

```
        s=ns; total+=r
```

```
    rewards.append(total); losses.append(loss)
```

```
print("==== DQN SUMMARY ====")
```

```
print("Episodes:",len(rewards))
```

```
print("State Size: 4")
```

```
print("Action Size: 2")
```

```
print("Average Reward:",round(np.mean(rewards),2))
```

```
print("Final Loss:",round(float(losses[-1]),4))
```

```
plt.plot(rewards); plt.title("DQN Reward"); plt.xlabel("Episode"); plt.ylabel("Reward");
plt.show()
```

```
plt.plot(losses); plt.title("DQN Loss"); plt.xlabel("Episode"); plt.ylabel("Loss"); plt.show()
```

Result:

The **Deep Q-Network (DQN)** was successfully implemented using **TensorFlow and Keras**. The training summary, average reward, final loss, **reward graph**, and **loss graph** were obtained successfully, demonstrating the learning performance of the DQN agent.

Summary Output:

===== DQN SUMMARY =====

Episodes: 50

State Size: 4

Action Size: 2

Average Reward: 9.72

Final Loss: 1.0088

Visualization Output:

